Análisis léxico-sintáctico y autómatas

Universidad de Caldas

Estructura de lenguajes

Ing. Rubén Darío García Guzmán

Agenda

- Introducción
- Función del analizador léxico
- Componentes léxicos, patrones y lexemas
- Especificación de los componentes léxicos
- Cadenas y lenguajes
- Operaciones aplicadas a lenguajes
- Expresiones regulares
- Definiciones regulares
- Reconocimiento de componentes léxicos
- Diagramas de transición
- Autómatas Finitos (AFN, AFD)
- Conversión de un AFD en un AFN

Introducción

- "*Una forma sencilla de crear un analizador léxico consiste en la construcción de un diagrama que ilustre la estructura de los componentes léxicos del lenguaje fuente, y después hacer la traducción "a mano" del diagrama a un programa para encontrar los componente léxicos.*"
- Alfred Aho, compiladores, principios, técnicas y herramientas.

Función del analizador léxico

- El análisis léxico o lineal (*scanner*), reconoce y clasifica todas las “palabras”, componentes léxicos o *tokens* de un código fuente.
- El analizador léxico lee el código fuente caracter a caracter y vá armando uno a uno los componentes léxicos que después entregará al analizador sintáctico, cada que este le de la orden de enviarle el siguiente token.
- El analizador léxico puede hacer tareas secundarias como: eliminar comentarios y espacios en blanco y relacionar los mensajes de error.

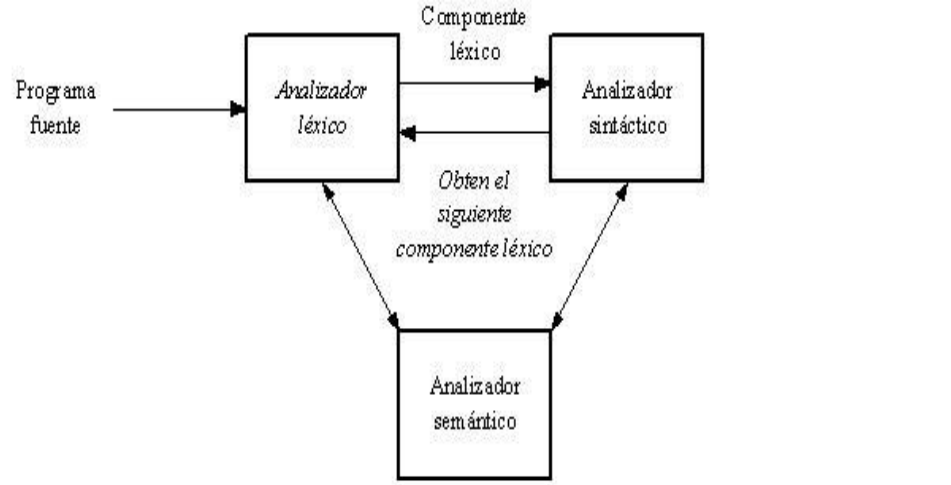


Figura 1. Interacción de un analizador léxico con el analizador sintáctico

Componentes léxicos, patrones y lexemas

- Estos tres nombres tienen significado específico en análisis sintáctico.
- En general hay un conjunto de cadenas de entrada para el cual se produce como salida el mismo *componente léxico*.
- Este conjunto de cadenas se construye mediante una regla llamada *patrón* asociado al componente léxico.
- Se dice que el patrón *concuerda* con cada cadena del conjunto.
- Un *lexema* es una secuencia de caracteres en el programa fuente con la que concuerda el patrón para un componente léxico.
- const PI=3.1416;
- la subcadena PI es un lexema para el componente léxico "Identificador"

Componente Léxico	Lexemas de Ejemplo	Descripción Informal del Patrón
const	const	const
if	if	if
relación	<,<=,=,<>,>,>=	< o <= o = o <> o > o >=
id	PI, cuenta, d2	letra seguida de letras o dígitos
num	3.1416, o, 6.02E-23	cualquier constante numérica
literal	"Hola mundo", 'c', TRUE	cualquier cadena entre "y "excepto "o cualquier caracter entre 'y',excepto 'o los valores TRUE o FALSE

Tabla 1. Ejemplo de componentes léxicos

Componentes léxicos, patrones y lexemas

- Los *componentes léxicos* se tratan como símbolos terminales de la gramática del lenguaje fuente.
- Los *lexemas* para el componente léxico que concuerdan con el patrón representan cadenas de caracteres en el programa fuente que se pueden tratar juntos como una unidad léxica.
- En la mayoría de lenguajes de programación, se consideran componentes léxicos las siguientes construcciones: ***palabras claves, operadores, identificadores, constantes, cadenas literales*** y ***signos de puntuación***.
- Un patrón es una regla que describe el conjunto de lexemas que pueden representar a un determinado componente léxico en los programas fuente.
- En muchos lenguajes, ciertas cadenas son *reservadas*, es decir, su significado está predefinido y el usuario no lo puede modificar.
- El analizador léxico debe distinguir entre una palabra clave y un identificador definido por el usuario.

Atributos de componentes léxicos

- Cuando concuerda con un componente léxico más de un patrón, el analizador léxico deberá proporcionar información adicional sobre el lexema concreto que concordó con las siguientes fases del compilador.

Componentes léxicos

Identificadores		
Lenguaje	Patrón	Ejemplos
C, Pascal, Java, C# y casi todos los lenguajes	Inician con letra y pueden ir seguidos de letras, dígitos o carácter subrayado	X1, r_23
PHP	Inician con $ y van seguidos de letras o dígitos	$x, $_POST
Prolog	Inician con letra, seguidas de létras o dígitos o _. Mayúsculas nombre de variable Minúsculas nombre de predicado	Nombre primo(X)

Componentes léxicos

Literales Numéricos		
Enteros, reales, reales con notación científica.		
Lenguaje	Patrón	Ejemplos
Todos	Enteros: Inician con signo opcional y va seguido de uno o más dígitos	-1223 99882
	Reales: Inician con signo opcional y van seguido de uno o más dígitos y opcional seguido de un punto y uno o más dígitos.	12.323 -44.53421
	Reales con notación científica: Inicia con signo opcional, y van seguidos de uno o más dígitos, seguido de punto “.”, seguido de uno o más dígitos, seguido de “E” seguido del signo + ó – opcional y seguido de uno o más dígitos.	-32.3e-19 7.4e+12 -9.1e10
Smalltalk, Scheme otros	Fraccion: inicia por signo o no, seguido de uno o más dígitos, seguido del slash “/” seguido de uno o más dígitos	43/28 2/78 -12/43

Componentes léxicos

Literales Caracter		
Lenguaje	Patrón	Ejemplos
C, java, C#, prolog, y otros	Un símbolo del alfabeto encerrado entre comillas sencillas ‘. Algunos caracteres especiales usan el símbolo backslash \	‘a’ ‘\n’ ‘Z’ ‘b’ ‘\t’
Pascal	Un símbolo del alfabeto encerrado entre comillas dobles “.	“a” “z” “b”
Smalltalk	Un símbolo del alfabeto precedido por el signo $	$a $$ $b
Scheme	Cualquier símbolo del alfabeto precedido por los símbolos #\	#\a #\b

Componentes léxicos

Literales Cadena		
Lenguaje	Patrón	Ejemplos
C, java, C#, prolog, scheme y otros	Cualquier conjunto de cero o más símbolos del alfabeto encerrados entre comillas dobles “. Prolog permite cualquier conjunto de símbolos que no contengan espacios e inicien por letra minúscula.	“Hola” “Mundo” “Adios Mundo”
Smalltalk, pascal	Cualquier conjunto de cero o más símbolos del alfabeto encerrados entre comillas sencillas ‘.	‘Hola Mundo’

Componentes léxicos

Literales Booleanos		
Lenguaje	Patrón	Ejemplos
Java, pascal, smalltalk, visual basic	Constante verdadera: true Constante falsa: false En smalltalk, en realidad son objetos	true, false
Scheme	Constante verdadera: #t Constante false: #f ó ()	#t, #f, ()
Lenguajes como C no tienen literales booleanas		

Componentes léxicos

Operadores Matemáticos		
Lenguaje	Patrón	Ejemplos
Casi todos los lenguajes	Suma +, Resta -, multiplicación *, División /	
C, java	Incremento ++, Decremento --; Modulo % No tiene operador de división entera	
Pascal	Modulo MOD, División entera DIV	
Visual Basic	Potenciación ^ División Entera \	

Componentes léxicos

Operadores Relacionales		
Lenguaje	Patrón	Ejemplos
Casi todos los lenguajes	Mayor que (>), Mayor o igual (>=), menor que (<), menor o igual (<=)	>, <, >=, <=
C, java	Diferente !=, igual ==	
Pascal, prolog, smalltalk	Igual en pascal, prolog y scheme =, diferente en pascal y prolog <>. En scheme es una función.	
Smalltalk y scheme, no tienen operador para diferente,		

Componentes léxicos

Operadores Booleanos		
Lenguaje	Patrón	Ejemplos
C, Java	And && ó &, Or || ó |, not !	
Pascal	And AND Or OR not NOT	
Lenguajes como prolog, scheme y smalltalk no tienen operadores relacionales, sino métodos o funciones para esto		

Componentes léxicos

Operador de Asignación		
Lenguaje	Patrón	Ejemplos
C, java, visual Basic, C#	=	a=10;
Pascal, smalltalk	:=	A:=10;
Prolog no tiene operador de asignación, pero tiene un operador similar que es :-. Scheme no tiene operadores de asignación		

Componentes léxicos

Operador de Concatenación		
Lenguaje	Patrón	Ejemplos
Java, C#	+	a=“Hola “; b=“Mundo”; printf(“%s”,a+b);
Pascal, C	, coma	a=‘Hola ‘; b=‘Mundo’; write(a,b);
Visual Basic	&	a=“Hola “ b=“Mundo” Print a & b
PHP	. punto	

Componentes léxicos

Separadores
. , ; .. ( ) { } [ ] : =

Componentes léxicos

Espacios en Blanco		
Lenguaje	Patrón	Ejemplos
En casi todos los lenguajes comprenden caracteres de Barra espaciadora, Nueva línea, retorno de carro y tabulador		
C, java	Nueva línea \n Retorno de carro \f Tabulador \t	
Visual Basic	Nueva Linea VBCR	
Otros lenguajes utilizan funciones como char(Ascii) de pascal que imprime el carácter de acuerdo con el codigo Ascii ingresado		

Componentes léxicos

Comentarios		
Existen comentarios de línea y de Bloque		
Lenguaje	Patrón	Ejemplos
C, java, muchos lenguajes	Línea // Bloque /* */ /** */	
Pascal	Bloque { } ó (* *)	
Prolog	Línea % Bloque /* */	
Visual Basic	Línea ‘	
Smalltalk	Bloque “ “	
Scheme	Línea ;	

Especificación de los componentes léxicos

- Las expresiones regulares son una notación importante para especificar patrones, cada patrón concuerda con una serie de cadenas, de modo que las expresiones regulares servirán como nombres para conjuntos de cadenas.

Cadenas y lenguajes

- El término ***alfabeto*** denota cualquier conjunto finito de símbolos.
- Ejemplos típicos de símbolos son las letras y los números.
- Por ejemplo, el alfabeto binario es representado por los símbolos {0,1}.
- Los códigos ASCII, EBCDIC o UNICODE, son ejemplos de alfabetos de computador.
- Una ***cadena*** sobre algún alfabeto, es una secuencia finita de símbolos tomados de ese alfabeto.
- La longitud de una cadena *s*, |*s*|, es el número de apariciones de símbolos en *s*.
- Por ejemplo, camino es una cadena de longitud seis(6).
- La cadena *vacía*, representada por $\in$, es una cadena especial, de longitud cero (0).

Cadenas y lenguajes

- El término *lenguaje* se refiere a cualquier conjunto de cadenas de un alfabeto fijo.
- Se pueden tener lenguajes abstractos como:
  - Ø (el conjunto *vacío)*
  - $\{\in\}$ y el conjunto que sólo contiene la cadena vacía
  - El conjunto de todos los programas de Pascal sintácticamente bien formados
  - El conjunto de todas las oraciones en inglés gramaticalmente correctas.
- Si *x* e *y* son cadenas, entonces la *concatenación* de *x* e *y*, (*xy)*, es la cadena que resulta de agregar *y* a *x*.
- La cadena vacía es el elemento de identidad que se concatena. Es decir, $s\in = \in s = s$.
- Si se considera la concatenación como "producto", cabe definir la "exponenciación" de cadenas de la siguiente forma.
  - Se define s^0 como $\in$, y para $i>0$ se define s^i como $s^{i-1}s$. Dado que $\in s$ es s, $s^1=s$. Entonces, $s^2=ss$, $s^3=sss$, etc.

Cadenas y lenguajes

TÉRMINO	DEFINICIÓN
Prefijo de *s*	Una cadena que se obtiene eliminando cero o más símbolos desde la derecha de la cadena *s*; por ejemplo, `ban` es prefijo de `bandera`.
Sufijo de *s*	Una cadena que se forma suprimiendo cero o más símbolos desde la izquierda de una cadena *s*; por ejemplo, `era` es sufijo de `bandera`.
subcadena de *s*	Una cadena que se obtiene suprimiendo un prefijo y un sufijo de *s*; por ejemplo, `ande` es subcadena de `bandera`. Todo prefijo y sufijo de *s* es una cadena de *s*, pero no toda subcadena de *s* es un prefijo o un sufijo de *s*. Para toda cadena *s*, tanto *s* como $\in$ son prefijos, sufijos y subcadenas de *s*.
Prefijo, sufijo o subcadena *propios* de *s*.	Cualquier cadena no vacía *x* que sea, respectivamente, un prefijo, un sufijo o subcadena de *s* tal que $s \neq x$.
subsecuencia de *s*	Cualquier cadena formada mediante la eliminación de cero o más símbolos no necesariamente contiguos de *s*; por ejemplo, `bada` es una subsecuencia de `bandera`.

Tabla 2. Términos de partes de una cadena

Operaciones aplicadas a lenguajes

- Algunas operaciones que se pueden aplicar a los lenguajes, son: la unión, la concatenación y la cerradura.
- También se puede extender el operador de "exponenciación" a los lenguajes, definiendo L^0 como $\{\in\}$, $\boldsymbol{L^i}$ como $\boldsymbol{L^{i-1}L}$. por tanto, $\boldsymbol{L^i}$ es ***L*** concatenado consigo mismo ***i -1*** veces.
- sea ***L*** el conjunto {A,B,...,Z,a,b,...,z} y ***D*** el conjunto {0,1,...,9}. ***L***, es el alfabeto que contiene el conjunto de letras mayúsculas, y ***D***, es el alfabeto que contiene el conjunto de los diez dígitos decimales.
- $\boldsymbol{L \cup D}$ *es el conjunto de letras y dígitos.*
- ***LD*** *es el conjunto de cadenas que consta de una letra seguida de un dígito.*
- $\boldsymbol{L^4}$ *es el conjunto de todas las cadenas de cuatro letras.*
- ***L**** *es el conjunto de todas las cadenas de letras, incluyendo* $\in$*, la cadena vacía.*
- $\boldsymbol{L(L \cup D)^*}$ *es el conjunto de todas las cadenas de letras y dígitos que comienzan por una letra.*
- ***D+*** *es el conjunto de todas las cadenas de uno o más dígitos.*

Operaciones aplicadas a lenguajes

OPERACIÓN	DEFINICIÓN
Unión de L y M, que se escribe $L \cup M$.	$L \cup M = \{ s \mid s$ está en L ó s está en $M \}$
Concatenación de L y M, que se escribe LM	$LM = \{ st \mid s$ está en L y t está en $M \}$
Cerradura de Kleene de L, que se escribe L^*	$L^* = \bigcup_{i=0}^{\infty} L^i$ L^* denota "cero o más concatenaciones de" L
Cerradura positiva de L, que se escribe L^+	$L^* = \bigcup_{i=1}^{\infty} L^i$ L^* denota "una o más concatenaciones de" L

Tabla 3. Definiciones de operaciones sobre lenguajes.

Expresiones regulares

- Normalmente, un identificador se define como una letra seguida de cero o más letras o dígitos **[L(L ∪ D)*]**.
- Existe una anotación llamada expresiones regulares, que permite definir de manera exacta conjuntos como éste, por ejemplo, la definición anterior quedaría como **letra (letra | digito)***
- La barra vertical (|) significa "o", los paréntesis se usan para agrupar subexpresiones, él asterisco (*) significa "cero o más casos de" la expresión entre paréntesis, y la yuxtaposición de **letra** con el resto de la expresión significa concatenación.
- Una expresión regular se construya partir de expresiones regulares más simples utilizando un conjunto de reglas definitorias.
- Cada expresión regular ***r*** representa un lenguaje ***L(r)***.
- Las reglas de definición especifican cómo se forma ***L(r)*** combinando de varias maneras los lenguajes representados por las subexpresiones de ***r***.

Expresiones regulares

- las siguientes son las reglas que definen las *expresiones regulares del alfabeto S:*
  - ∈ es una expresión regular definida por {∈}; es decir, el conjunto que contiene la cadena vacía.
  - si ***a*** es un símbolo de *S*, entonces ***a*** es una expresión regular designada por ***{a}***; por ejemplo, el conjunto que contiene la cadena ***a***.
  - suponiendo que ***r*** y es ***s*** sean expresiones regulares representadas por los lenguajes ***L(r)*** y ***L(s)***, entonces,
    - a) ***(r)|(s)*** es una expresión regular representada por ***L(r)*** ∪ ***L(s)***.
    - b) ***(r)(s)*** es una expresión regular representada por ***L(r)L(s)***.
    - c) ***(r)**** es una expresión regular representada por ***(L(r)*)***.
    - d) ***(r)*** es una expresión regular representada por ***(L(r))***.
- el operador unario (* ó +) tiene la mayor precedencia y es asociativo por la izquierda.
- La concatenación tiene la segunda mayor precedencia y es asociativa por la izquierda.
- | tiene la menor precedencia y es asociativo por la izquierda.
- *(a)|((b)*(c))* es equivalente a *a|b*c*.

Expresiones regulares

- Existen varias leyes algebraicas que obedecen las expresiones regulares y pueden ser utilizadas para transformar las expresiones regulares a formas equivalentes.
- En la tabla siguiente se muestran algunas leyes que se cumplen para las expresiones regulares r, s y t.

AXIOMA	DESCRIPCIÓN
$r\|s = s\|r$	| es conmutativo
$r\|(s\|t)$	| es asociativo
$(rs)t = r(st)$	la concatenación es asociativa
$r(s\|t) = rs \mid rt$ $(s\|t)r = sr \mid tr$	la concatenación es distributiva sobre |
$\in r = r$ $r \in = r$	$\in$ es el elemento de identidad para la concatenación
$r^* = (r \mid \in)^*$	la relación entre * y $\in$
$r^{**} = r^*$	* es idempotente

Tabla 4. Propiedades algebraicas de las expresiones regulares.

Definiciones regulares

- Se puede dar nombres a las expresiones regulares y definir expresiones regulares utilizando dichos nombres como si fueran símbolos.
- si S es un alfabeto de símbolos básicos, entonces una *definición regular* es una secuencia de definiciones de la forma

  d1 → *r1*
  d2 → *r2*
  . . .
  dn → *rn*
- donde cada *di* es un nombre distintivo y cada *ri* es una expresión regular sobre los símbolos de S ∪ {*d1, d2, ..., di-1*}.
- Al limitar cada *ri* a los símbolos de S y a los nombres previamente definidos, se puede construir una expresión regular en S para cualquier *ri*, reemplazando una y otra vez los nombres de las expresiones regulares por las expresiones que designan.

- el conjunto de identificadores

  letra → A | B | . . . | Z | a | b | . . . | z
  digito → 0 | 1 | . . . | 9
  id → **letra**(**letra**|**digito**)*
- Los números sin signo son cadenas como 58394, 23.54, 6.335E3.

  digito → 0 | 1 | . . . | 9
  digitos → **digito digito***
  fraccion_optativa → .**digitos** | **Î**
  exponente_optativo → (E(+|-| **Î**) **digitos**) | **Î**
  num → **digitos fraccion_optativa exponente_optativo**

Abreviaturas en la notación

- Uno o más casos = +. rr*=r+
- Cero o un caso = ?. r|∈ = r?
- Clases de caracteres. [s1 s2 s3]. a|b|c = [abc] = [a-c]

  digito → [0-9]
  digitos → **digito**+
  fraccion_optativa → (.**digitos**)?
  exponente_optativo → (E(+|-)? **digitos**)?
  num → **digitos fraccion_optativa exponente_optativo**

Reconocimiento de componentes léxicos

- La gramática

  prop → **if** *expr* **then** *prop*
  | **if** *expr* **then** *prop* **else** *prop*
  | ∈

  expr → *término* **oprel** *término*
  | *término*

  término → **id**
  | **núm**

- Con los símbolos terminales

  if → if
  then → then
  else → else
  oprel → <|<=|=|<>|>|>=
  id → letra (letra | digito)*
  núm → dígito+ (. dígito+)?(E(+ | -)?dígito+)?

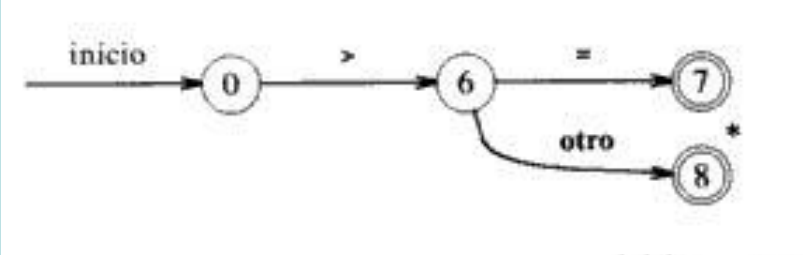
- Para eliminar espacios en blanco

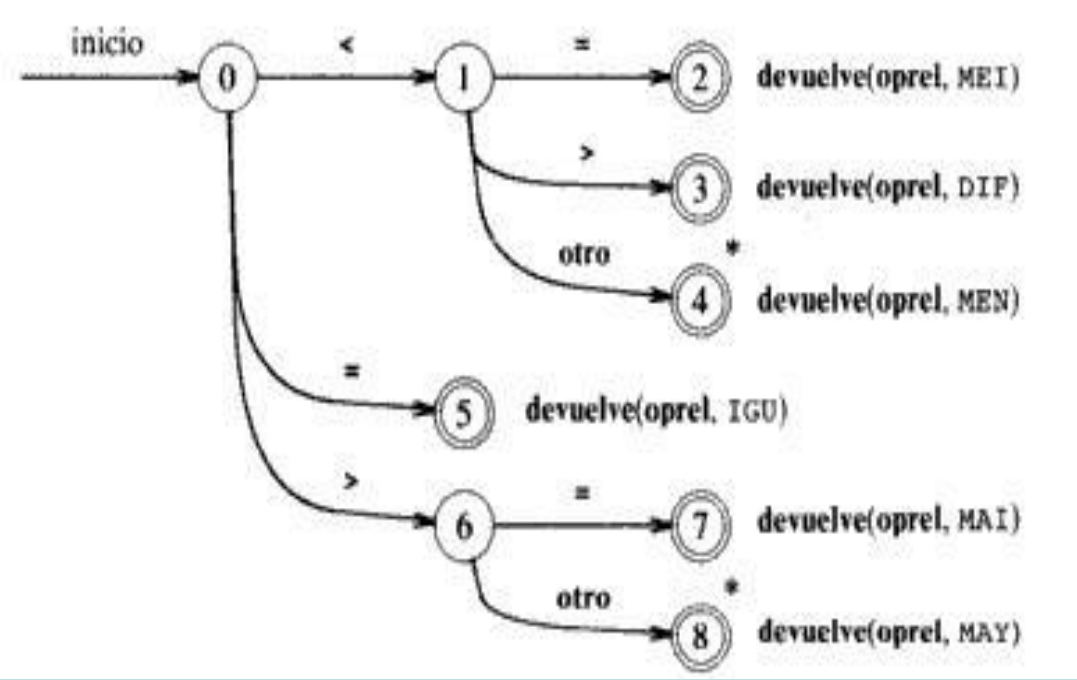
  delim → blanco | tab | líneanueva
  eb → delim+

Diagramas de transición

- Como paso intermedio en la construcción de un analizador léxico, primero se produce un diagrama de flujo estilizado, llamado *diagrama de transiciones*.
- Los diagramas de transiciones representan las acciones que tienen lugar cuando el analizador léxico es llamado por el analizador sintáctico para obtener el siguiente componente léxico.
- Las posiciones en un diagrama de transición se representan con un círculo y se llaman *estados*.
- Los estados se conectan mediante flechas, llamadas *aristas*.
- Las aristas que salen del estado *s* tienen etiquetas que indican los caracteres de entrada que pueden aparecer después de haber llegado el diagrama de transición al estado *s*.
- Si los diagramas de transiciones son *deterministas*; ningún símbolo puede concordar con las etiquetas de dos aristas que salgan de un estado.
- Cuando el diagrama de transición es *no determinista*, facilita las cosas al diseñador del analizador léxico y, con las herramientas adecuadas, también al encargado de su implantación.
- Un estado se etiqueta como el *estado de inicio*; es en el estado inicial del diagrama de transición donde reside el control cuando se empieza a reconocer un componente léxico.
- Ciertos estados pueden tener acciones que se ejecutan cuando el flujo del control alcanza dicho estado.
- Al entrar en un estado se lee el siguiente carácter de entrada. Si hay una arista del estado en curso de ejecución cuya etiqueta concuerde con ese carácter de entrada, entonces se va al estado apuntado por la arista. De otro modo, se indica un fallo.

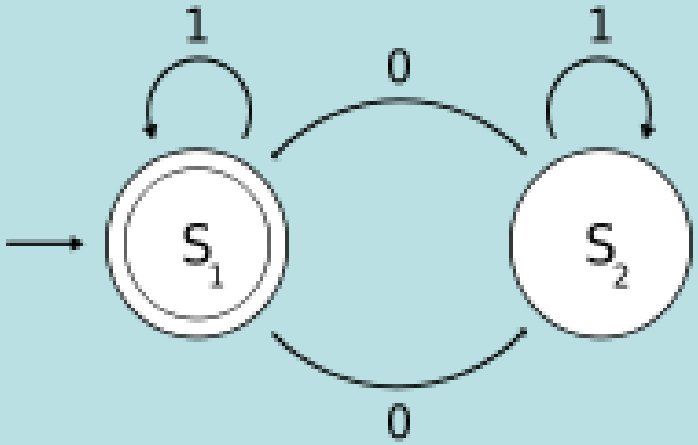
Diagramas de transición





Autómatas finitos

- Un *reconocedor* de un lenguaje es un programa que toma como entrada una cadena x y responde "**si**" si x es una frase del programa o "**no**", si no lo es.
- Se compila una expresión regular en un reconocedor, construyendo un diagrama de transiciones generalizado llamado Autómata Finito.
- Se clasifican en deterministas (AFD) y no deterministas (AFN), siendo los no deterministas los que pueden salir de un estado por dos transiciones distintas marcadas con la misma entrada.



Autómatas Finitos - Introducción

- Ambos tipos de autómatas pueden reconocer con precisión los conjuntos regulares.
- La diferencia está entre espacio y tiempo.
- Mientras que un AFD ofrece reconocedores más rápidos, un AFN es menor que su equivalente AFD.

Autómatas Finitos no Deterministas

- Un AFN es un modelo matemático formado por:
  1. Un conjunto de *estados S*.
  2. Un conjunto de símbolos de entrada Σ (el alfabeto de símbolos de entrada).
  3. Una función de transición *mueve* que transforma pares estado-símbolo en conjuntos de estados.
  4. Un estado s_0 que se considera el *estado de inicio* (o inicial).
  5. Un conjunto de estados *F* considerados como *estados de aceptación* (o finales).
- Un AFN se representa mediante un grafo de transiciones en el que los nodos son estados y las aristas etiquetadas representan la función de transición.

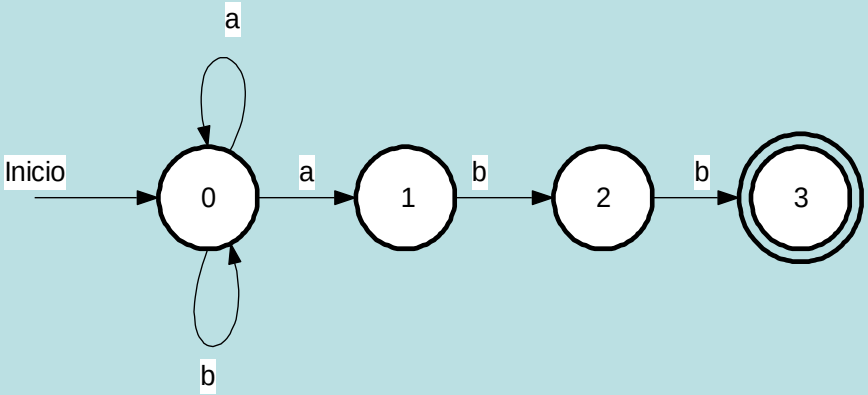


Fig. 1 Un autómata finito no determinista

Autómatas Finitos no Deterministas

- El grafo se parece a un diagrama de transiciones, pero el mismo carácter puede etiquetar dos o más transiciones fuera de un estado.
- Además las aristas pueden etiquetarse con el símbolo especial ∈ y con símbolos de entrada.
- La representación del grafo de transiciones se puede hacer con una tabla de transiciones.

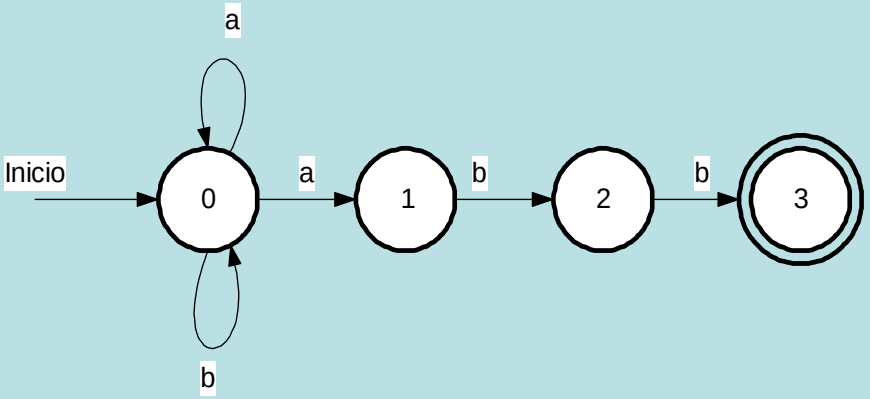


Fig. 1 Un autómata finito no determinista

Estado	Símbolo de Entrada	
	a	b
0	{0,1}	{0}
1	-	{2}
2	-	{3}

Autómatas Finitos no Deterministas

- Un *AFN* acepta una cadena de entrada x si, y sólo si, hay algún camino en el grafo de transiciones desde el estado de inicio a algún estado de aceptación, de forma que las etiquetas de las aristas a lo largo de dicho camino acepten x.
- Se representa el camino mediante una secuencia de transiciones de estados llamada *movimientos*.
- El lenguaje definido por un AFN es el conjunto de cadenas de entrada que acepta.

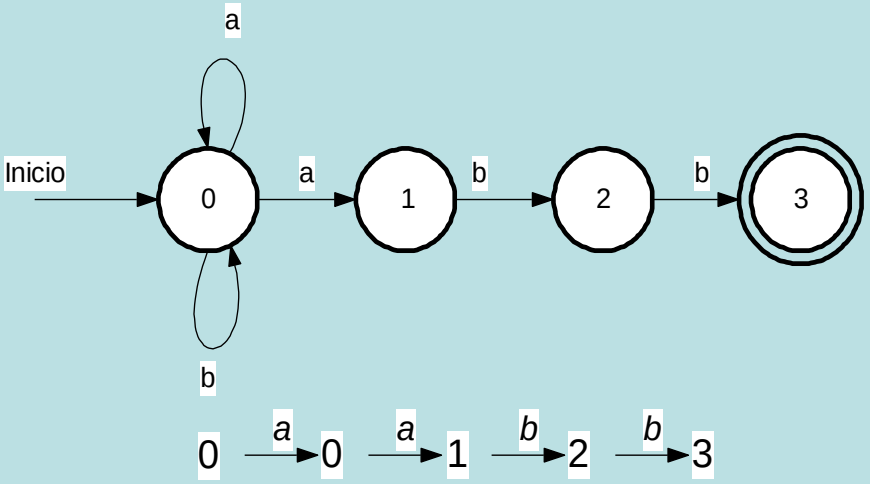


Fig. 2 Movimientos realizados para aceptar la cadena de entrada *aabb*

Autómatas Finitos Deterministas

- Un AFD es un caso especial de AFN en el cual:
  1. Ningún estado tiene una transición ∈, es decir, una transición con entrada ∈. Un conjunto de *estados S*.
  2. Para cada estado *S* y cada símbolo de entrada *a* hay a lo sumo, una arista etiquetada *a* que sale de *S*.
- El algoritmo muestra como simular el comportamiento de un AFD

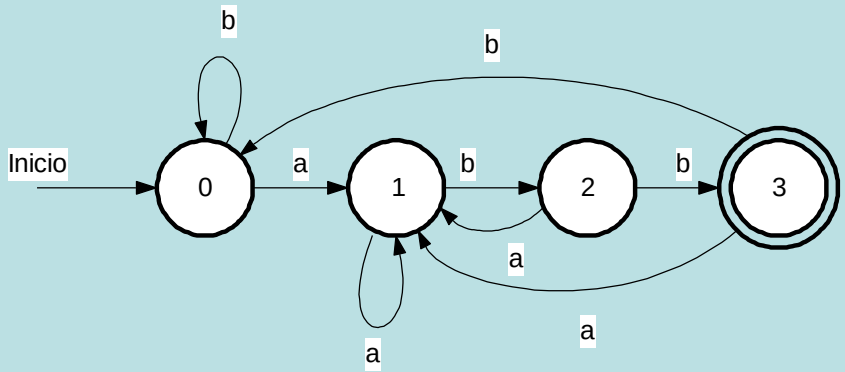


Fig. 3 *un* AFD que acepta (a|b)*abb

Entrada: Una cadena *x* que termina con un carácter fin de archivo ***eof***. Un AFD *D* con estado inicial S_0 y un conjunto *F* de estados de aceptación.

Salida: La respuesta "*si*", si *D* acepta *x*; "*no*" en caso contrario.

Método: la función *mueve(s,c)* da el estado al cual hay una transición desde el estado *s* en un carácter de entrada *c*. La función *Sgtecar* devuelve el siguiente carácter de la cadena de entrada *x*.

```
Funcion AFD()
    s<-s0
    c<-sgtecar
    Mientras c<>eof haga
          s<-mueve(s,c)
          c<-sgtecar
    fin_mientras
    si s está en F entonces
          devolver "si"
    sino
          devolver "no"
Fin
```